БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет прикладной математики и информатики
Кафедра технологий программирования

Лабораторная работа №2
Коммуникация в многоконтейнерных системах

По дисциплине «Непрерывное интегрирование и сборка программного обеспечения»

Методические указания по выполнению лабораторной работы

Подготовила:
Давидовская М. И.,
Ст. преподаватель кафедры ТП

Минск, 2026 г.

Содержание

Цель работы....3
Задачи работы....3
Краткие теоретические сведения....3
Задания....4
Методические указания....4
Критерии оценивания....4
Содержание отчета....4
Задание 1. Разработка простейшего распределенного приложения….6
Варианты к заданию 1....7
Задание 2. Асинхронная коммуникация в распределенных системах на основе брокера сообщений....10
Упражнение 2.1....10
Упражнение 2.2....10
Упражнение 2.3....10
Задание 3. Асинхронная коммуникация в распределенных системах на основе потоковой системы....11
Контрольные вопросы....13

Цель работы

Проектирование распределенных систем и реализация коммуникации между их компонентами

Задачи работы

1. Реализовать простую распределенную систему
2. Реализовать распределенную систему с асинхронным обменом на основе брокера сообщений
3. Оформить отчёт в формате Markdown и опубликовать результаты.

Краткие теоретические сведения

Издатель (publisher) или «производитель» (producer) — это приложение (или экземпляр приложения), которое публикует (создает) сообщения. Одно и то же приложение может также использовать сообщения и, таким образом, одновременно быть потребителем (consumer).

Термин «потребитель» (Consumer) в разных предметных областях может иметь разные значения. В общем, в контексте обмена сообщениями и потоковой передачи данных **потребитель** — это приложение (или экземпляр приложения), которое использует и подтверждает сообщения. Одно и то же приложение может также публиковать сообщения и, таким образом, одновременно быть «издателем» (publisher).

В протоколах обмена сообщениями используется концепция постоянной подписки на доставку сообщений. **Подписка** — это один из терминов, обычно используемых для описания такой сущности. **Потребитель** (consumer) — другой ключевой термин. В протоколах обмена сообщениями, поддерживаемых RabbitMQ, используются оба термина, но в документации RabbitMQ предпочтение отдается последнему.

Потребитель — это подписка на рассылку сообщений, которая должна быть зарегистрирована до начала рассылки и может быть отменена приложением.

RabbitMQ — это посредник обмена сообщениями. Он принимает сообщения от издателей, маршрутизирует их и, если есть очереди для

маршрутизации, сохраняет их для использования или немедленно доставляет потребителям при их наличии.

Издатели публикуют сообщения по назначению, которое зависит от протокола. В AMQP 0-9-1 издатели публикуют материалы в точках обмена (exchange). В AMQP 1.0 публикация осуществляется по ссылке (link). В MQTT издатели публикуют материалы в разделах (topics). Наконец, STOMP поддерживает множество типов адресатов: разделы, очереди, точки обмена по протоколу AMQP 0-9-1.

Опубликованное сообщение должно быть перенаправлено в очередь (раздел и т.д.). В очереди (разделе) могут быть онлайн потребители. Когда сообщение перенаправлено в очередь и в сети есть потребитель, который может принимать доставку, сообщение будет отправлено получателю.

Попытка публикации в несуществующей очереди (теме) приведет к возникновению исключения с кодом 404.

Потребители получают сообщения из очередей. При добавлении нового потребителя, при условии, что в очереди уже есть готовые сообщения, доставка начнется немедленно.

Целевая очередь может быть пустой на момент регистрации потребителя. И в таком случае первые доставки будут осуществляться при постановке новых сообщений в очередь.

Задания

Методические указания

Все результаты лабораторной работы должны быть опубликованы в git-репозитории, ссылка на который доступна в курсе «Непрерывное интегрирование и сборка программного обеспечения»

Критерии оценивания

Для групп 10-12 выполнить задания №1-3 (все задания) и ответы на все контрольные вопросы, для группы 13 — задание 1 и упражнение 2.2 из задания №2 и ответы на все контрольные вопросы.

Содержание отчета

1. Цель работы.
2. Вариант задания.

3. Код приложений, конфигурационных файлов.

4. Ответы на контрольные вопросы.

Отчет должен быть опубликован в git-репозитории на github. Все результаты лабораторной работы должны быть опубликованы в git-репозитории, ссылка на который доступна в курсе «Непрерывное интегрирование и сборка программного обеспечения».

В файле Readme проекта на github должна быть ссылка на отчёт. Отчет опубликовать во внешнем хранилище или в репозитории в каталоге /docs. Если в лабораторной работе необходимо написать программу/ы, то отчёт должен результаты тестов по каждой программе и ответы на контрольные вопросы.

Пример оформления файла Readme может быть таким:

```
# Overview

Report on LabRabota2.

# Usage

// Заменить <<link>> и <<folder>> на соответствующие ссылки/названия
To check, please, preview report by <<link>> and source files
in <<folder>>.

# Author

Your name and group number.

# Additional Notes

// СКОПИРОВАТЬ И ВСТАВИТЬ ССЫЛКУ НА СВОЙ РЕПОЗИТОРИЙ,НАПРИМЕР
https://github.com/maryiad/lab3-task1-gr16-david
```

Каждая лабораторная работа содержит тексты задач и контрольные вопросы, ответы на которые проверяются преподавателем при приёме работы у студента.

Выполнение студентом лабораторной работы и сдача её результатов преподавателю происходит следующим образом:

1. Студент выполняет разработку программ.

2. В ходе разработки студент обязан следовать указаниям к данной задаче (в случае их наличия). Исходные тексты программ следует разрабатывать в соответствии с требованиями к оформлению, приведёнными в приложении.

3. Студент выполняет самостоятельную проверку исходного текста каждой разработанной программы и правильности её работы, а также свои знания по теме лабораторной работы.

Задание 1. Разработка простейшего распределенного приложения

1. Изучить материалы по монтированию каталогов (связанных томов монтирования — **bind mounts**) из хост-системы в контейнер docker:
   - https://docs.docker.com/engine/storage/ — типы хранилищ;
   - https://docs.docker.com/engine/storage/bind-mounts/ — монтирование связанных томов (bind mounts).
2. Согласно вашему варианту разработать два приложения так, чтобы результат первого являлся исходными данными для второго. Язык разработки может быть любым, например Python, или C/C++, или Java, или другой.
3. В каталоге репозитория для лабораторной работы 2 создать ветку **task1** от ветки **main** и каталог **task1**. Каталог **task1** должен содержать файлы **Readme**, **.gitignore**, **.dockerignore**, **compose.yaml** (**docker-compose.yml**).
4. В каталоге **task1** создать каталоги **data**, **result-1**, **result, worker1**, **worker2**.
5. Для связи двух приложений использовать следующую схему, где символ «**.**» означает путь относительно текущего каталога, т. е. **task1**:
   - Каталог **./data** должен быть примонтирован в каталог **/var/data** для программы 1. — *Оттуда будут браться исходные данные.*
   - Каталог **./result-1** должен быть примонтирован в каталог /var/result для программы 1. — *Туда будут складываться промежуточные данные.*
   - Каталог **./result-1** должен быть примонтирован в каталог **/var/data** для программы 2. — *Оттуда будут браться промежуточные результаты.*
   - Каталог **./result** должен быть примонтирован в каталог **/var/result** для программы 2. — *Туда будут складывать результаты финальной обработки.*

6. В каталогах **worker1**, **worker2** должны содержаться исходный файлы приложения, файл **.gitignore** (при необходимости), и конфигурационный файл Dockerfile для сборки образов. Dockerfile должен содержать комментарии к блокам инструкций.
7. Собрать образы docker для каждого из созданных приложений, описав конфигурацию в Dockerfile.
8. Создать файл **compose.yaml** (**docker-compose.yml**) в каталоге **task1** для сборки и запуска приложений с помощью команды

   ```
   docker compose up –build
   ```

   Требования к файлу **compose.yaml** (**docker-compose.yml**):
   - Два сервиса.
   - Каждый соответствует программам согласно варианту.
   - Объявлена директива **build** для каждого сервиса.
   - Монтирование каталогов с данными.
   - Описание зависимостей сервисов одного от другого.
9. Опубликовать ветку **task1**, выполнить pull request и слить с веткой **main**. Ветку **task1** не удалять.

Варианты к заданию 1

1. **Вариант задания соответствует порядковому номеру в группе**.
2. Содержимое исходных файлов — целые числа.
3. Результат выполнения программы 2 необходимо дополнительно вывести на экран.

Варианты для программ 1 и 2

№	Программа 1	Программа 2
1	Ищет в каталоге **/var/data** самый большой по объёму файл и перекладывает его в **/var/result/data.txt**	Сохраняет произведение первого и последнего числа из файла **/var/data/data.txt** в **/var/result/result.txt**
2	Ищет в каталоге **/var/data** файл с наибольшим количеством строк и перекладывает его в	Ищет наибольшее число из файла **/var/data/data.txt** и сохраняет его вторую

	/var/result/data.txt	степень в **/var/result/result.txt**
3	Формирует файл **/var/result/data.txt** из первых строк всех файлов каталога **/var/data**	Ищет наименьшее число из файла **/var/data/data.txt** и сохраняет его третью степень в **/var/result/result.txt**
4	Формирует файл **/var/result/data.txt** так, что каждая строка файла - количество строк в файлах из каталога **/var/data**.	Ищет наибольшее число из файла **/var/data/data.txt** и сохраняет количество таких чисел из последовательности в **/var/result/result.txt**.
5	Формирует файл **/var/result/data.txt** так, что каждая строка файла - количество символов в именах файлов из каталога **/var/data**.	Ищет наименьшее число из файла **/var/data/data.txt** и сохраняет количество таких чисел из последовательности в **/var/result/result.txt**
6	Ищет в каталоге **/var/data** файл с самым коротким названием и перекладывает его в **/var/result/data.txt**.	Ищет наименьшее число из файла **/var/data/data.txt** и сохраняет его третью степень в **/var/result/result.txt**
7	Берёт из каталога **/var/data** случайный файл и перекладывает его в **/var/result/data.txt**.	Ищет наибольшее число из файла **/var/data/data.txt** и сохраняет количество таких чисел из последовательности в **/var/result/result.txt**.
8	Ищет в каталоге **/var/data** самый большой по объёму файл и перекладывает его в **/var/result/data.txt**	Ищет наименьшее число из файла **/var/data/data.txt** и сохраняет количество таких чисел из последовательности в

		/var/result/result.txt
9	Ищет в каталоге **/var/data** файл с наибольшим количеством строк и перекладывает его в **/var/result/data.txt**	Ищет наибольшее число из файла **/var/data/data.txt** и сохраняет его вторую степень в **/var/result/result.txt**
10	Формирует файл **/var/result/data.txt** из первых строк всех файлов каталога **/var/data**	Ищет наименьшее число из файла **/var/data/data.txt** и сохраняет его третью степень в **/var/result/result.txt**
11	Формирует файл **/var/result/data.txt** так, что каждая строка файла - количество строк в файлах из каталога **/var/data**.	Ищет наименьшее число из файла **/var/data/data.txt** и сохраняет его третью степень в **/var/result/result.txt**
12	Формирует файл **/var/result/data.txt** так, что каждая строка файла - количество символов в именах файлов из каталога **/var/data**.	Ищет наименьшее число из файла **/var/data/data.txt** и сохраняет количество таких чисел из последовательности в **/var/result/result.txt**
13	Ищет в каталоге **/var/data** файл с самым коротким названием и перекладывает его в **/var/result/data.txt**.	Сохраняет произведение первого и последнего числа из файла **/var/data/data.txt** в **/var/result/result.txt**
14	Берёт из каталога **/var/data** случайный файл и перекладывает его в **/var/result/data.txt**.	Ищет наибольшее число из файла **/var/data/data.txt** и сохраняет его вторую степень в **/var/result/result.txt**

Задание 2. Асинхронная коммуникация в распределенных системах на основе брокера сообщений

1. Получить изменения из репозитория для ветки **main**, в которую были опубликованы изменения ветки **task1** с помощью **pull request**.
2. В репозитории создать ветку **task2** от **main** и каталоги **task2-1**, **task2-2**, **task2-3** для каждого из упражнений.
3. Выполнить упражнения 2.1, 2.2 и.2.3.
4. Опубликовать исходный код приложений, конфигурационных файлов **Dockerfile**, **compose.yaml** (**docker-compose.yml**) в репозиторий.
5. Продемонстрировать использование .gitignore, .dockerignore.
6. Оформить файл **Readme** для ветки **task2**.
7. Опубликовать ветку **task2**, выполнить pull request и слить с веткой **main**. Ветку **task2** не удалять.

Упражнение 2.1

1. Изучить статью RabbitMQ для аналитика: практический ликбез.
2. Создать учетную запись на сервисе CloudAMQP.
3. Реализовать пример из статьи RabbitMQ для аналитика: практический ликбез.
4. В каталоге **task2-1** опубликовать фрагменты json и описание хода выполнения задания.

Упражнение 2.2

1. Изучить статью Setup RabbitMQ with Docker Compose.
2. Настроить контейнер докер и продемонстрировать получение доступа к интерфейсу администрирования.
3. Пройти уроки 1, 2 и 3 из RabbitMQ Tutorials. **Группы 10-12** — на любом языке программирования, **группа 13** — язык программирования Python. Код из уроков хранится локально на хост-системе. Контейнер создавать не надо.
4. При защите лабораторной работы продемонстрировать работу брокера сообщений на основе примеров из уроков 1-3.

Упражнение 2.3

1. Изучить статьи:
   - Технология RPC и ее реализацию с помощью Rabbit MQ и Pika.

- Глава 1. Как оживает Rabbit.
- От монолитного кода к сервисам с помощью RabbitMQ и Pika.
- Как запускать RabbitMQ в Docker.
- Installing and using RabbitMQ with Docker.
- Пишем продюсер и консумер для RabbitMQ на Python в Google Colab.

2. Настроить распределенную систему, состоящую из контейнеров с Rabbit-MQ, Producer (приложение публикует сообщения в очередь) и Consumer (приложение читает из очереди), взяв за основу пример из любой статьи.
3. В качестве хранилища в каждом приложении использовать связанную точку монтирования (bind mount).
4. Для каждого из контейнеров создать Dockerfile и для управления инфраструктурой настроить файл compose.yaml (docker-compose.yml).
5. Включить в состав инфраструктуры инфраструктуру для локального ИИ-агента из задания 6 лабораторной работы 1 и продемонстрировать его применение при работе с кодом и конфигурационными файлами.
6. Продемонстрировать использование консольных утилит rabbitmqctl, rabbitmqadmin.
7. Опубликовать конфигурационные файлы и файлы с исходным кодом приложений.

Задание 3. Асинхронная коммуникация в распределенных системах на основе потоковой системы

1. Получить изменения из репозитория для ветки **main**, в которую были опубликованы изменения ветки **task2** с помощью **pull request**.
2. В репозитории создать ветку **task3** от **main** и каталог **task3**.
3. Изучить статьи:
   - Обзор UI-инструментов для мониторинга и управления кластерами Apache Kafka.
   - Как запустить Apache Kafka с GUI на офисном ноутбуке и Windows: Docker в WSL и Настройка портов Docker-контейнеров компонентов Kafka для запуска на офисном ноутбуке и Windows.
   - Настройка Kafka в Docker.

4. Изучить пример инфраструктуры ниже и реализовать инфраструктуру на основе Apache Kafka с UI на основе AKHQ, Offset Explorer (ранее Kafka Tool) или UI for Apache Kafka с простейшим производителем (producer) и потребителем (consumer) как в статье Настройка Kafka в Docker.
5. Опубликовать исходный код приложений, конфигурационных файлов **Dockerfile**, **compose.yaml** (**docker-compose.yml**) в репозиторий.
6. Продемонстрировать использование .gitignore, .dockerignore.
7. Дополнить файл **Readme** для ветки **task3**.
8. Опубликовать ветку **task3**, выполнить pull request и слить с веткой **main**. Ветку **task3** не удалять.

Пример инфраструктуры на основе Apache Kafka, Kraft, AKHQ использует режим KRaft (без отдельного Zookeeper) и веб-интерфейс AKHQ:

```yaml
version: '3.8'

services:
  kafka:
    image: confluentinc/cp-kafka:7.6.0
    container_name: kafka_local
    ports:
      - "9092:9092"
    environment:
      # Настройки узла и KRaft режима
      KAFKA_NODE_ID: 1
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: 'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT'
      KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092'
      KAFKA_OFFERS_INTER_BROKER_LISTENER_NAME: 'PLAINTEXT'
      KAFKA_PROCESS_ROLES: 'broker,controller'
      KAFKA_CONTROLLER_QUORUM_VOTERS: '1@kafka:29093'
      KAFKA_LISTENERS: 'PLAINTEXT://0.0.0.0:29092,CONTROLLER://0.0.0.0:29093,PLAINTEXT_HOST://0.0.0.0:9092'
      KAFKA_INTER_BROKER_LISTENER_NAME: 'PLAINTEXT'
      KAFKA_CONTROLLER_LISTENER_NAMES: 'CONTROLLER'
      KAFKA_LOG_DIRS: '/tmp/kraft-combined-logs'
      # Идентификатор кластера (сгенерирован заранее)
      CLUSTER_ID: 'MkU3OEVBNTcwNTJENDM2Qk'

  akhq:
    image: tchiotludo/akhq:latest
    container_name: akhq_local
    ports:
      - "8080:8080"
    environment:
      AKHQ_CONFIGURATION: |
        akhq:
          connections:
            docker-kafka-cluster:
              properties:
                bootstrap.servers: "kafka:29092"
    depends_on:
      - kafka
```

Контрольные вопросы

1. Что такое обмен сообщениями (messaging) и какие проблемы в архитектуре ПО он решает?

2. Что такое брокер сообщений? Какова его роль в системе?

3. Перечислите и охарактеризуйте основные шаблоны (паттерны) обмена сообщениями.

4. Дайте определение RabbitMQ.

5. Какой прототокол лежит в основе RabitMQ?

6. В RabbitMQ сообщения, принятые от приложения-продюсера (издателя), записываются …

7. Объясните, что такое точка обмена (exchange).

8. Что такое Binding в RabbitMQ?

9. Дайте определение Routing Key в Rabbit MQ.

10. Опишите по шагам процесс маршрутизации сообщения в RabbitMQ (от продюсера до потребителя).

11. Если необходимо просто распараллелить обработку сообщений, принятых от продюсера (издателя) по нескольким потребителям, какой тип обменника надо выбрать в RabbitMQ?

12. Какие типы точек обмена (exchanges) существуют в RabbitMQ и в чем их принципиальные отличия?

13. Обычно приложение-продюсер отправляя сообщение в RabbitMQ, отправляет его ……

14. Как предотвратить потребление устаревших данных из RabbitMQ?

15. По какому принципу LIFO или FIFO устроены очереди в RabbitMQ, в которые записываются сообщения?

16. Для чего предназначен Server (Broker/Node) в RabbitMQ.

17. Опишите назначение Virtual Host (Vhost) в RabbitMQ.

18. Что такое подтверждение сообщений (acknowledgments /ack, nack, reject) и зачем оно необходимо?

19. Чем отличаются долговечные (durable) и временные (transient) очереди и сообщения? Как это влияет на перезагрузку брокера?

20. Что такое Dead Letter Exchange (DLX) и какие сценарии его использования?

21. Что такое Priority Queue в RabbitMQ и как она работает?

22. Дайте определение Apache Kafka. Какие основные задачи она решает и чем ее философия отличается от классических брокеров?

23. Опишите архитектуру Kafka. Что такое Broker, Topic и Partition&

24. Что такое Consumer Group (группа потребителей) и как она влияет на масштабирование и распараллеливание чтения?

25. Что такое реплика (Replica)? Чем роль Leader отличаается от роли Follower?

26. Что такое ZooKeeper (или подсистема KRaft в новых версиях) и какова его роль в архитектуре Kafka?

27. Что такое Kafka Streams и как он отличается от написания Consumer-приложений?

28. В чем заключаются фундаментальные архитектурные отличия между RabbitMQ и Apache Kafka (модель хранения, маршрутизация, состояние сообщений)?

29. Приведи примеры, в каких сценариях предпочтительнее использовать RabbitMQ, а в каких — Apache Kafka.

30. Можно ли использовать RabbitMQ и Apache Kafka в одной архитектуры? Если — да, то за какие зоны ответственности каждый из них будет отвечать?